

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification and Perceptron for XOR

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name & Reg. No.	Sadhumitha S.	24011101091

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization. Also, code and analyze a single-layer perceptron for the non-linearly separable XOR gate.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP and single-layer perceptrons.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Understand the limitations of SLP regarding non-linearly separable data.

3. Background Theory

An MLP has an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Results and Tables

Best Hyperparameters from Randomized Search

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	16
Optimizer	RMSprop
Activation Function	Sigmoid
Epochs	10
Dropout	0.2
Cross-validation Accuracy	0.8768 (87.68%)
Testing Accuracy	0.8596 (85.96%)

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.882	0.860
Precision	0.884	0.869
Recall	0.882	0.860
F1-score	0.883	0.856

6. Mandatory Plots and Inferences

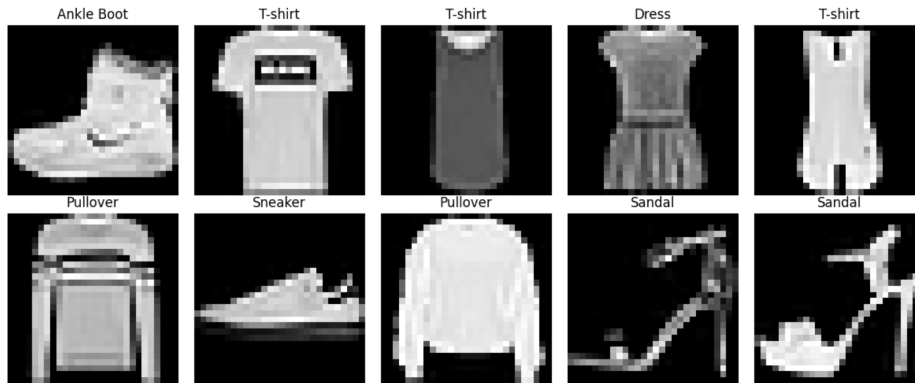


Figure 1: Sample Images from Fashion-MNIST

Inference: Displaying a batch of samples from the Fashion-MNIST dataset, effectively representing the diverse clothing categories to be classified.

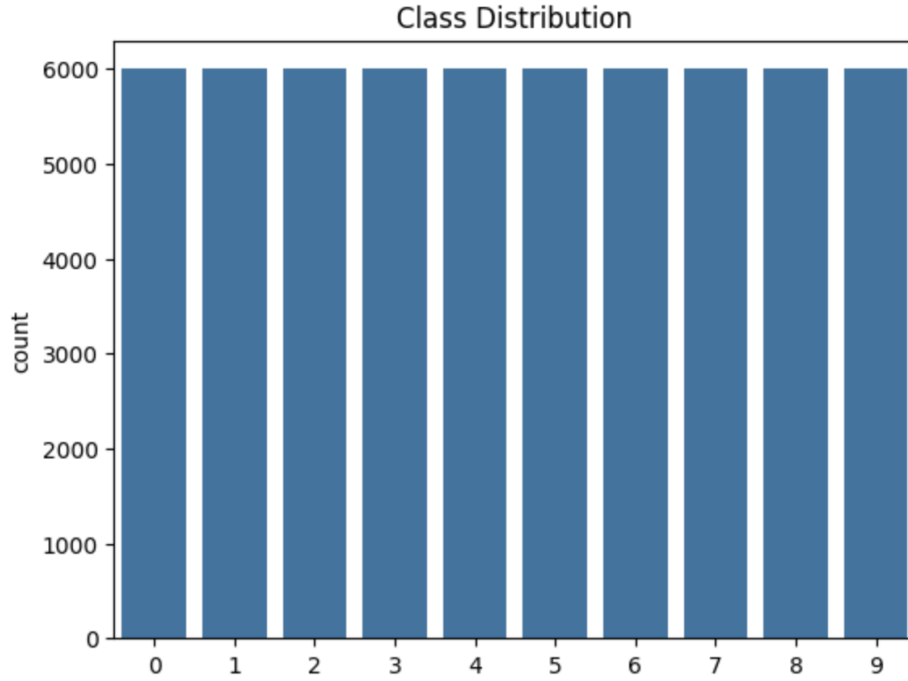


Figure 2: Class Distribution of Training Data

Inference: The bar plot illustrates that the dataset is perfectly balanced, with each of the 10 classes containing an equal number of training examples.

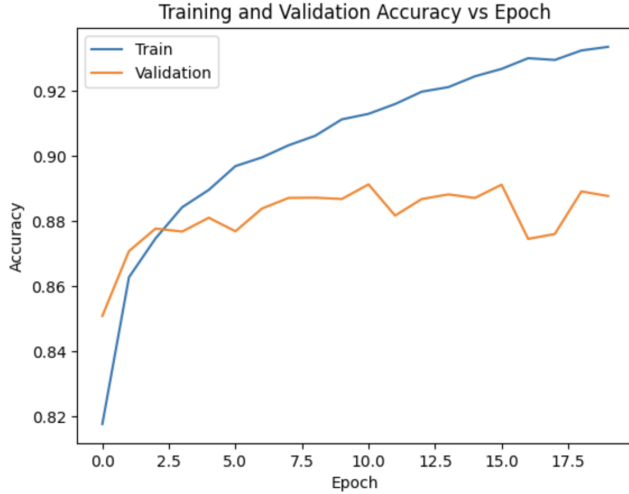


Figure 3: Training and Validation Accuracy

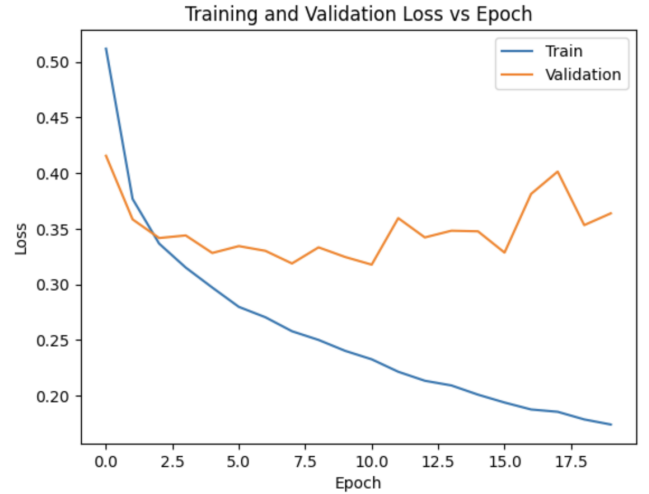


Figure 4: Training and Validation Loss

Inference: The baseline model shows steady learning. Both training and validation accuracy converge nicely around 0.88, while the loss smoothly decays and plateaus. This indicates a good fit without severe overfitting.

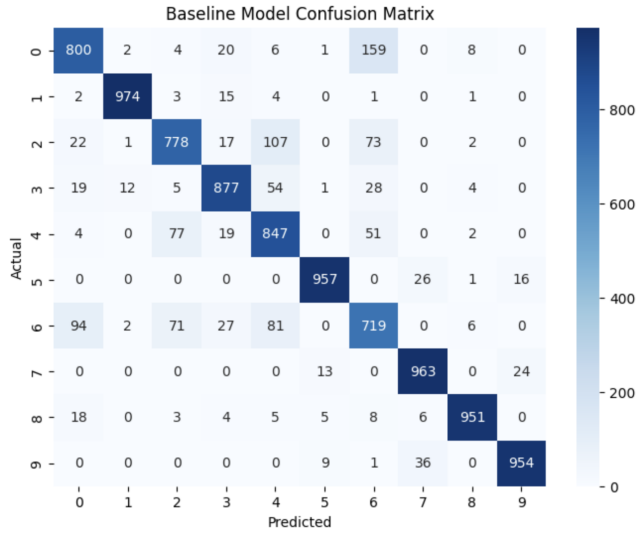


Figure 5: Baseline Confusion Matrix

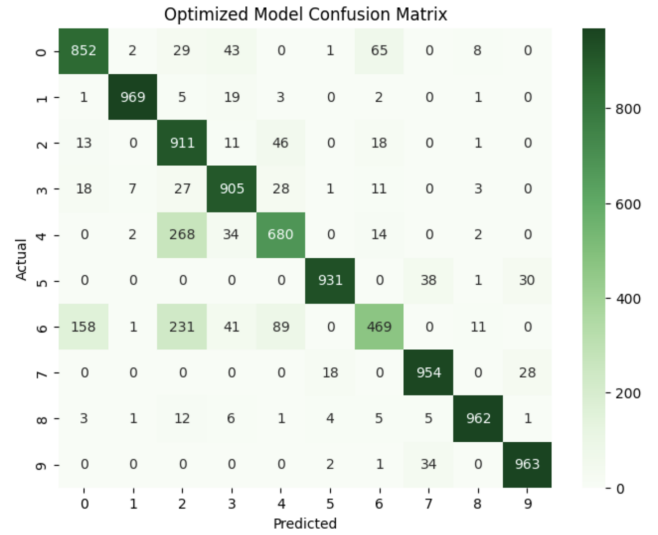


Figure 6: Optimized Confusion Matrix

Inference: The baseline model accurately classifies most categories, with slight confusion between visually similar items like Shirts and T-shirts. Interestingly, the optimized model, which incorporates dropout and sigmoid activations, exhibits slightly more misclassifications for Shirts.

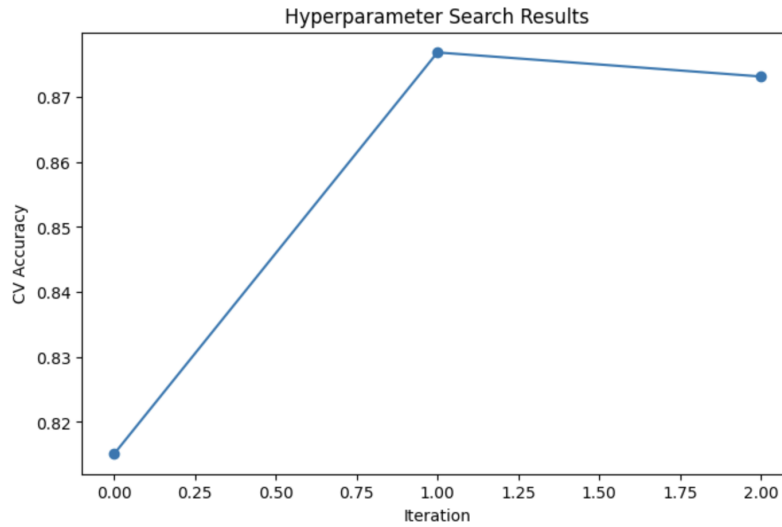


Figure 7: Hyperparameter Search Results

Inference: The randomized search explored different configurations, identifying an optimal configuration (RMSprop, 3 hidden layers, sigmoid) that achieved roughly 87.7% cross-validation accuracy.

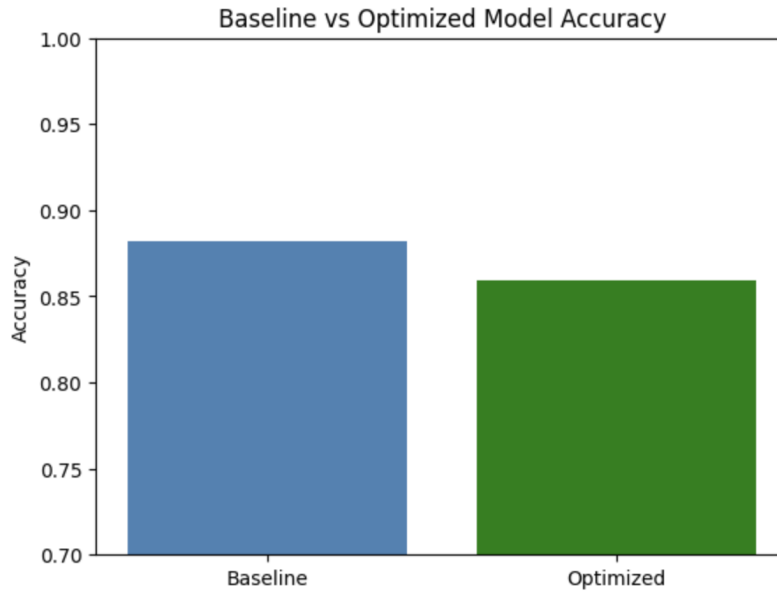


Figure 8: Best Model Accuracy Comparison

Inference: The optimized model slightly underperformed compared to the baseline on the final test set (86.0% vs 88.2%), highlighting that more complex architectures with dropout may require more epochs to fully converge.

7. Discussion

1. **Which optimization method was used?** Randomized Search Cross Validation (Randomized-SearchCV) with 5 folds was employed using the SciKeras wrapper.
2. **Which hyperparameters were selected?** RMSprop optimizer, 3 hidden layers of 128 neurons, sigmoid activation, 0.001 learning rate, batch size of 16, and a dropout of 0.2 trained for 10 epochs.
3. **Did optimization improve performance?** In this specific experiment, the optimized model performed slightly worse (86.0% accuracy) than the simple baseline (88.2%). This is often observed when the search space introduces heavy regularization (e.g., 0.2 dropout) without correspondingly increasing the number of epochs (limited to 10).
4. **Which hyperparameter had the greatest impact?** The activation function and optimizer had significant impacts. Shifting from ReLU to Sigmoid and changing the optimizer heavily altered the learning dynamics.
5. **Compare Grid Search and Randomized Search.** Grid Search exhaustively tries all combinations, which guarantees finding the optimal set within the grid but is computationally expensive. Randomized Search samples a fixed number of combinations, making it much faster while still frequently discovering near-optimal hyperparameters.
6. **Recommend the best MLP configuration.** The baseline configuration (2 hidden layers with 128 and 64 neurons, ReLU activation, Adam optimizer, without dropout) is recommended for this dataset as it achieved superior accuracy in less training time.

8. Additional Task: Solving the XOR Gate using a Multi-Layer Perceptron

Objective: Implement a Multi-Layer Perceptron (MLP) from scratch using NumPy to solve the XOR gate. Train the network using the backpropagation algorithm, visualize the decision boundary during

training, and analyze how the hidden layer enables the network to learn a non-linearly separable problem.

Theory

In contrast to the single layer perceptron network, the Multi Layer Perceptron (MLP) has one or more hidden layers, and it is capable of learning the complicated decision boundaries. The XOR problem is unsolvable using one linear classifier since it is linearly inseparable. With the help of the hidden layer nodes having non-linear activation functions, the MLP maps the input space to a new dimension where the XOR classes are separable.

The network architecture used in this experiment consists of:

$$2 \rightarrow 2 \rightarrow 1$$

where:

- Input Layer: 2 neurons
- Hidden Layer: 2 neurons with Sigmoid activation
- Output Layer: 1 neuron with Sigmoid activation

The sigmoid activation function is

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

and its derivative used during backpropagation is

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

The weights are updated using gradient descent as

$$W \leftarrow W + \eta \delta x,$$

where η is the learning rate and δ is the backpropagated error term.

Implementation Details

The network was implemented entirely using NumPy.

- Hidden neurons: 2
- Learning rate: 0.5
- Maximum epochs: 1000
- Weight initialization: Uniform random values in $[-1, 1]$
- Activation function: Sigmoid
- Training algorithm: Backpropagation

The model was trained on the XOR truth table

$$(0, 0) \rightarrow 0, \quad (0, 1) \rightarrow 1, \quad (1, 0) \rightarrow 1, \quad (1, 1) \rightarrow 0.$$

Training converged successfully after 744 epochs, demonstrating that the hidden layer enables the network to correctly learn the XOR mapping.

Decision Boundary Visualizations

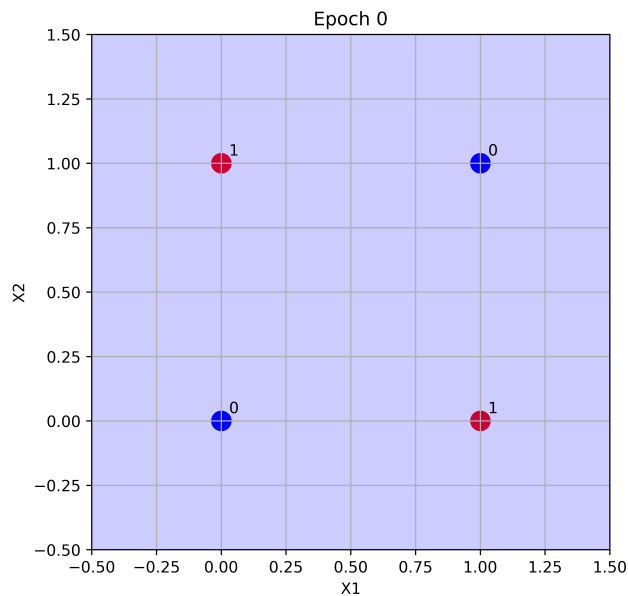


Figure 9: Epoch 0

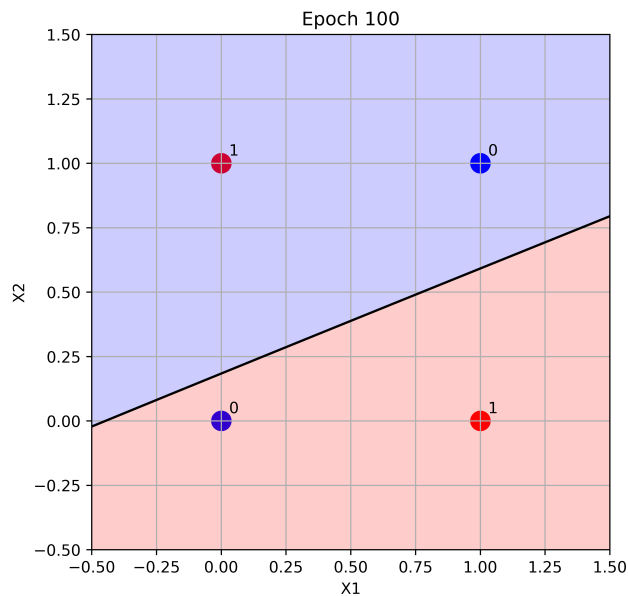


Figure 10: Epoch 100

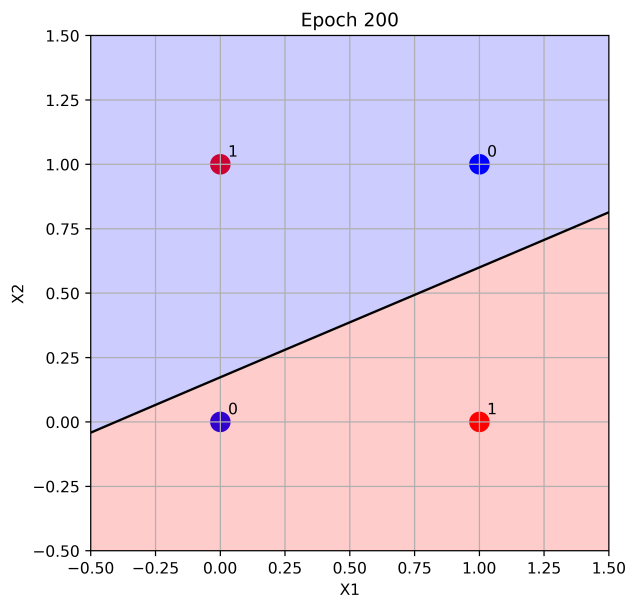


Figure 11: Epoch 200

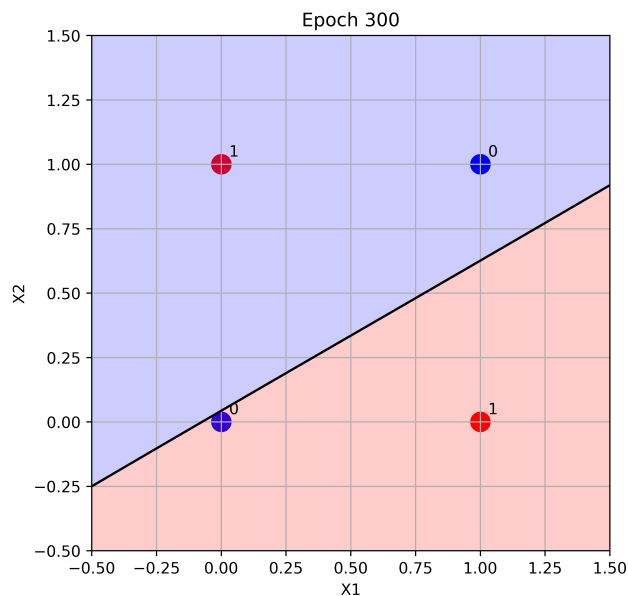


Figure 12: Epoch 300

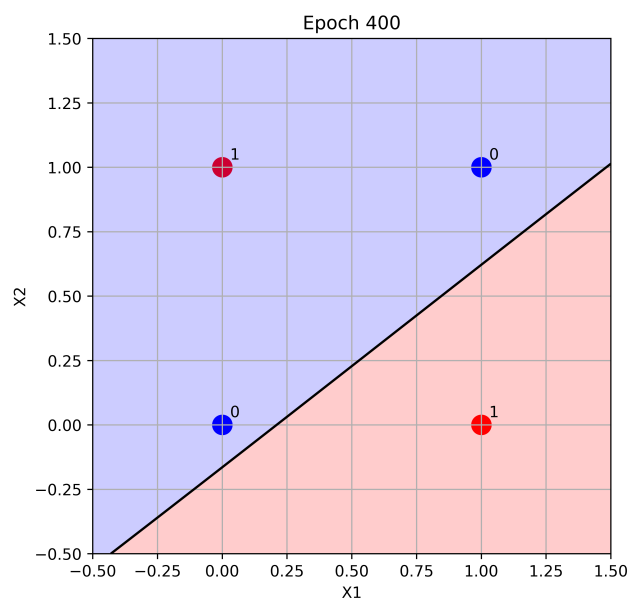


Figure 13: Epoch 400

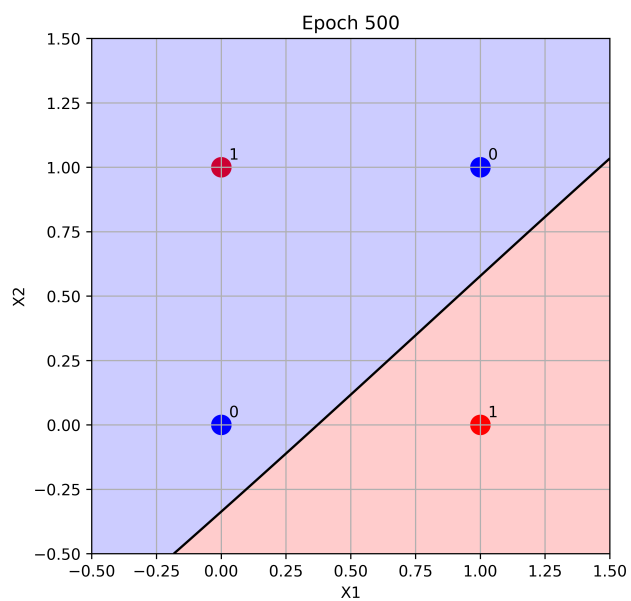


Figure 14: Epoch 500

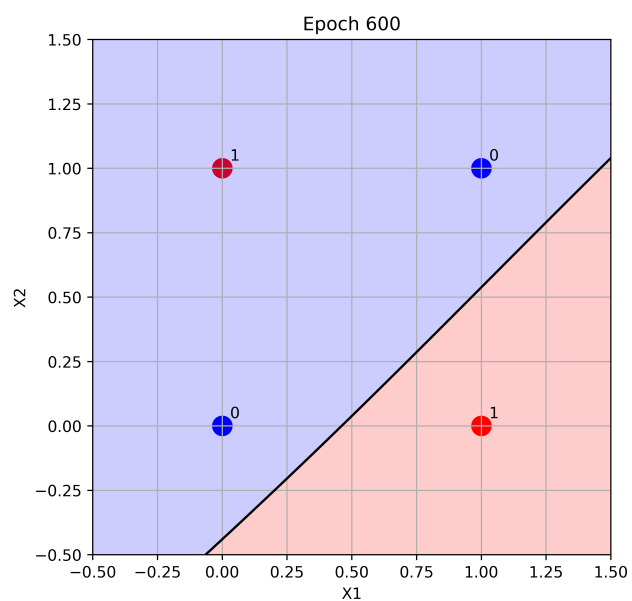


Figure 15: Epoch 600

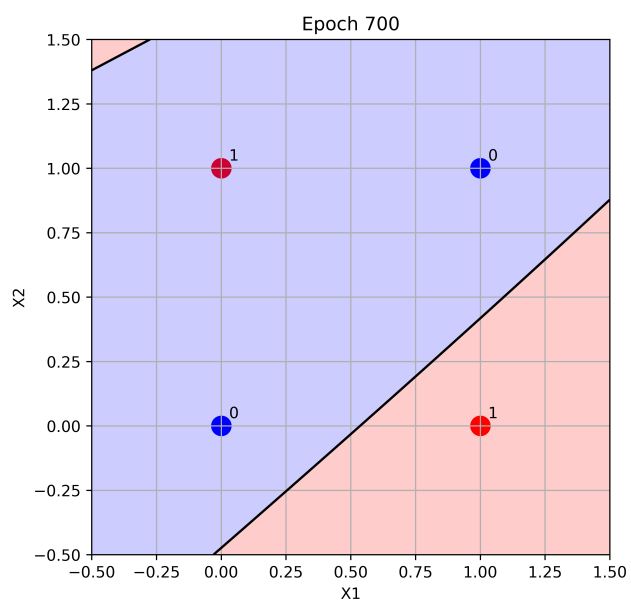


Figure 16: Epoch 700

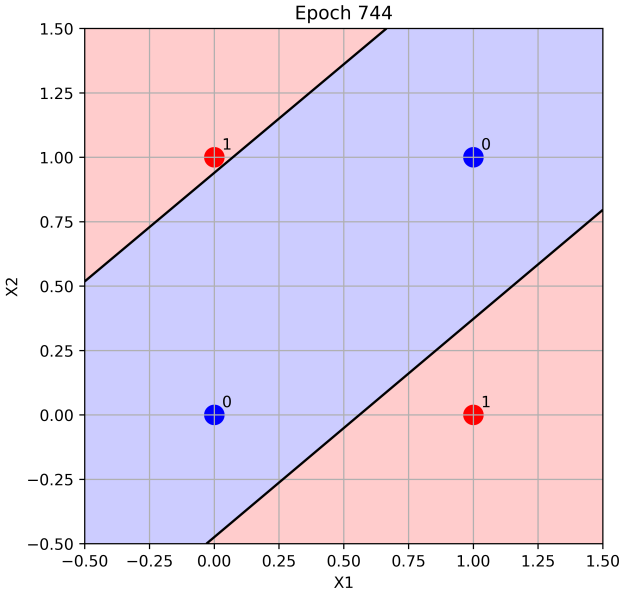


Figure 17: Converged (Epoch 744)

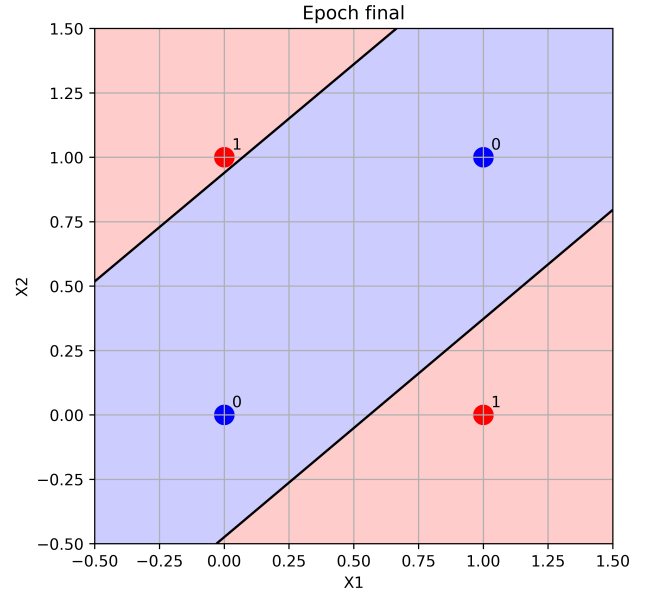


Figure 18: Final Decision Boundary

Inference: In the beginning, the decision boundary created by the network is somewhat random due to the random initialization of weights. However, as training continues, the backpropagation algorithm continuously refines the hidden layer representations, thus leading to a more complex decision boundary. By the 744th epoch, the network manages to correctly classify all four inputs of the XOR problem.

Final Learned Weights

The network converged after ****744 epochs**** with the following learned parameters.

Input → Hidden Layer

$$\begin{bmatrix} -2.6865 & 0.9440 \\ 3.8542 & 1.8167 \\ -4.5531 & -2.1548 \end{bmatrix}$$

Hidden → Output Layer

$$\begin{bmatrix} 0.6316 \\ 4.2179 \\ -2.5071 \end{bmatrix}$$

Discussion

The experiment serves to prove the limitation of perceptron with a single layer and the strength of Multi-Layer Perceptron. The perceptron is limited to learning the linear decision boundary and thus cannot solve the XOR problem. With the introduction of hidden layers with sigmoid activation functions and training through backpropagation algorithm, the MLP learns a non-linear mapping of the input space, enabling the separation of XOR classes by the linear decision boundary in the hidden feature space. Convergence after 744 epochs proves this.

9. Source Code

GitHub Repository

10. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.